Python Basics · Day 1 of 30 · about 60 minutes

Hello, Python! Python as a calculator

🎯 Today's goal

By the end of this hour you will:

- run your very first Python code
- make Python show messages with `print()`
- use Python as a super-calculator
- read an error message without panicking

🤔 Why this matters

Every data scientist and AI engineer started exactly here. Python is the most popular language for data science and AI because it reads almost like English. Think of it as giving very precise instructions to a very fast, very literal assistant: it does **exactly** what you write — no more, no less.

🧭 How this page works

The grey boxes are **code boxes**: real Python code, with what it prints shown just underneath. To run and change the code yourself, open this day's `index.html` in a browser. Each box has a ▶ **Run** button.

```
print("Hello, world!")
```

> `Hello, world!`

💬 Showing messages with `print()`

`print()` tells Python: "show this on the screen". Text must go inside quotes — either `"double"` or `'single'` quotes work. Text in quotes is called a **string**.

```
print("My name is Python.")
print('I love data.')
print("Each print starts on a new line.")
```

```
My name is Python.
I love data.
Each print starts on a new line.
```

You can print several things at once by separating them with commas. Python puts a space between them for you:

```
print("I am", 25, "years old")
```

```
I am 25 years old
```

Notice that `25` has **no quotes**. That's because it's a **number**, not text. This difference matters, as you'll see in a moment.

🧮 Python as a calculator

Python understands maths. The symbols are called **operators**:

Operator	Meaning	Example	Result
`+`	add	`7 + 3`	`10`
`-`	subtract	`7 - 3`	`4`
`*`	multiply	`7 * 3`	`21`
`/`	divide	`7 / 2`	`3.5`
`//`	whole-number divide	`7 // 2`	`3`
`%`	remainder	`7 % 2`	`1`
`**`	power	`2 ** 3`	`8`

```
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(2 ** 3)
```

```
10
4
21
3.5
3
1
8
```

Handy tip: if the *last line* of a box is a calculation, Python shows its value even without `print()` — just like a calculator.

```
100 * 1.2
```

```
120.0
```

📐 Order of operations

Python follows the same rules you learned at school: brackets first, then powers, then multiply/divide, then add/subtract.

```
print(2 + 3 * 4)     # multiply first: 2 + 12
print((2 + 3) * 4)   # brackets first: 5 * 4
```

```
14
20
```

See the `#` symbols? Everything after a `#` is a **comment**. Python ignores comments — they are notes for humans. Good programmers use them to explain *why* they did something.

🔤 Text vs numbers

Quotes change everything. With quotes, `"5"` is the *character* 5, not the number five:

```
print(5 + 5)       # numbers: adds them
print("5" + "5")   # strings: glues them together!
```

```
10
55
```

Gluing strings together is called **concatenation**. It's useful for building messages:

```
print("data" + " " + "science")
print("ha" * 3)
```

```
data science
hahaha
```

🐞 Errors are your friends

Everyone — even experts — gets errors every day. An error is just Python saying "I don't understand this line". It tells you **which line** and **what kind** of problem. This box has a missing closing quote on purpose:

💡 This example has a mistake on purpose. Run it and read the error.

```
print("Hello)
```

> ⚠️ **SyntaxError**
>
> Python couldn't read this code. Look for a missing quote, bracket, colon or comma near the line shown.
>
> ```
>   line 1
>     print("Hello)
>           ^
> SyntaxError: unterminated string literal (detected at line 1)
> ```

Python reports a `SyntaxError`: the code isn't written in a form Python can read. Here's another common one — mixing text and numbers:

💡 This example has a mistake on purpose. Run it and read the error.

```
print("I am " + 25 + " years old")
```

> ⚠️ **TypeError**
>
> You combined things that don't fit together, like text + a number.
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     print("I am " + 25 + " years old")
>           ~~~~~~~~^~~~
> TypeError: can only concatenate str (not "int") to str
> ```

A `TypeError` means you tried to combine things that don't fit together. Fix it with commas instead: `print("I am", 25, "years old")`.

How to read an error: look at the **last line** first. It names the error type and gives a short explanation. Then look at the line number above it.

⚠️ Common mistakes

- Forgetting quotes around text: `print(Hello)` → Python thinks `Hello` is a name it should already know.
- Mismatched quotes: `print("Hi')` → the start and end quotes must be the same kind.
- Capital letters: `Print("Hi")` doesn't work. Python is **case-sensitive**: `print` ≠ `Print`.
- Forgetting the brackets: `print "Hi"` was old Python 2 style and no longer works.

✅ Recap

- `print(...)` shows things on the screen.
- Text goes in quotes (a **string**); numbers don't.
- `+ - * / // % **` do maths; brackets control the order.
- `#` starts a comment.
- Errors are normal: read the last line first.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Say hello

Make Python print exactly this message: `Hello, Python!`

```
# Write your code below
```

Hints:

1. Use the `print()` function.
2. Text must be inside quotes: `print("...")`
3. Check the capital H and P, the comma and the exclamation mark.

2. Fix the bug

This code has a mistake and won't run. Fix it so it prints: `I am learning data science`

```
print("I am learning data science)
```

Hints:

1. Run it first and read the error message.
2. Every opening quote needs a matching closing quote.

3. Seconds in a day

A day has 24 hours, each hour has 60 minutes, and each minute has 60 seconds. Use Python to **calculate** and print how many seconds are in one day. Let Python do the multiplying — don't type the answer yourself!

```
# Multiply hours × minutes × seconds
```

Hints:

1. The multiply operator is `*`
2. `print(24 * 60 * 60)`

4. Shopping bill

You buy **3 notebooks** costing **2.5** each and **2 pens** costing **1.25** each. Print the total cost using one calculation.

```
# notebooks + pens
```

Hints:

1. The notebooks cost `3 * 2.5`
2. Add the cost of the pens: `2 * 1.25`
3. `print(3 * 2.5 + 2 * 1.25)`

5. Sharing pizza

17 pizza slices are shared fairly between 5 friends. On the **first line**, print how many whole slices each friend gets. On the **second line**, print how many slices are left over. Use the `//` and `%` operators.

```
# Line 1: slices each friend gets

# Line 2: slices left over
```

Hints:

1. `//` gives the whole-number part of a division.
2. `%` gives the remainder.
3. You need two `print()` lines: one with `17 // 5` and one with `17 % 5`.

➡️ Tomorrow

Variables: giving names to values so Python can remember them — the first step towards working with real data.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Say hello

```
print("Hello, Python!")
```

2. Fix the bug

```
print("I am learning data science")
```

3. Seconds in a day

```
print(24 * 60 * 60)
```

4. Shopping bill

```
print(3 * 2.5 + 2 * 1.25)
```

5. Sharing pizza

```
print(17 // 5)
print(17 % 5)
```